

Shot Prediction Model & Feature Analysis

NAME: Sadou Barry | **NUMBER:** 8226

1. Executive Summary

This project develops an explainable, highly performant probabilistic model to predict shot attempt success in anonymized NBA tracking data. Moving from a baseline Logistic Regression to a tuned XGBoost gradient boosting architecture reduced validation Log-Loss from 0.63951 down to 0.62922.

Because individual player identities were anonymized in the dataset, model performance relies entirely on capturing geometric spatial relationships, closeout mechanics, shooter movement, and tactical game context. Evaluating these situational mechanics yields an optimal probability estimate for any given shot attempt across the court.

2. Feature Engineering & Basketball Domain Logic

Rather than relying strictly on raw dataset metrics, engineered features were designed around real world basketball observations, drawing heavily from consistent film study and watching games regularly across the season. In particular, observing defensive closeout rotations during the 2026 Knicks and Spurs Finals series and studying off ball movement mechanics exemplified by my favorite player Steph Curry directly shaped the feature pipeline.

Defender Pressure Ratio ($\text{def_pressure_ratio} = \text{closestdefdist} / (\text{dist_to_rim} + 1)$):

The interaction feature `def_pressure_ratio`, defined as closest defender distance divided by rim distance plus one, emerged as the single most influential feature in the XGBoost model with approximately 37 percent feature importance. Measuring raw defender distance in isolation creates blind spots because four feet of separation on a perimeter jumper yields a clean, rhythm look, whereas four feet of separation directly at the rim constitutes severe contest and altered shot trajectory. Watching perimeter rotations consistently shows that defenders recovery speed matters most relative to the shooters proximity to the rim, making this spatial ratio far more informative than standalone linear distance.

Shooter Speed & Moving Jumpers ($\text{is_moving_jumper} = \text{shotttype} == \text{"jumper"} \ \& \ \text{shooterspeed} > 5$):

The feature `is_moving_jumper`, which isolates jump shots taken while traveling at speeds greater than five feet per second, captures the severe mechanical difficulty of off balance shooting. Watching elite off ball movement, exemplified by my favorite player Steph Curry, highlights how rare it is for shooters to maintain perfect alignment, squared shoulders, and vertical elevation while curling around screens at top speed. High speed pull ups or fading jumpers off screens significantly alter release timing compared to stationary spot up looks, drastically impacting shot trajectory and entry angle into the rim.

Shot Clock Context & Forced Looks (is_late_clock)

The situational indicator is_late_clock isolates shot attempts taking place with four seconds or fewer on the shot clock during set defense possessions. Watching late clock offensive possessions reveals that these attempts rarely resemble structured offense; instead, they are usually forced, contested, off balance looks taken against a primed defensive shell. Differentiating these forced bailout looks from early clock transition opportunities allows the model to properly discount make probability when time runs out.

Non-Linear Distance Decay (dist_to_rim_sq)

The non-linear term dist_to_rim_sq accounts for the exponential decay in shooting accuracy as distance from the basket increases. While spatial geometry provides linear distance, shooting variance expands rapidly beyond the paint, particularly in the deep mid range and behind the three point arc where arc angle, depth control, and fatigue have a far greater compounding effect on shot outcomes.

3. Modeling Methodology & Validation Strategy

A strict 80/20 train and validation split was established with a set random seed (set.seed(30)) to evaluate all feature modifications against Log-Loss on unseen data.

Model	Validation Log-Loss	Key Drivers and Takeaways
Baseline Logistic Regression	0.63951	Established simple additive linear relationships across spatial coordinates and basic flags.
Tuned XGBoost (eta = 0.05)	0.62922	Captured critical non-linear interactions between defender distance, shooter speed, shot angle, and rim proximity.

Hyperparameter Configuration

The final model utilized gradient boosted decision trees (binary:logistic) trained with a conservative learning rate (eta = 0.05) to ensure smooth gradient convergence without overfitting the training set. Tree depth was capped at max_depth = 6, while subsample = 0.8 and colsample_bytree = 0.8 were applied to introduce stochastic feature selection across individual iterations.

4. Limitations & Future Work

While the XGBoost model does a solid job reading basic tracking numbers, any hoop-head watching film knows that raw numbers on a stat sheet only tell part of the story. There are a few real world basketball variables that would take this model from a solid baseline to an elite scouting tool.

Player Level Baseline Identifiers

Without player names or IDs attached to the tracking data, the model treats every single player on the court the exact same way. In reality, a contested twenty eight foot jumper taken by Steph Curry is a great offense possession, whereas that same look taken by a non shooter is a defensive win. Knowing who is actually taking the shot and factoring in their career shooting averages would immediately make these make probabilities way more realistic.

Sequential Streak Dynamics

Basketball is a game of runs and confidence, but the dataset looks at every shot as an isolated event in a vacuum. It completely misses the hot hand effect or a player feeling rhythm after hitting two shots in a row. Tracking recent performance within a game, like how many shots a player made in their last few possessions or how hot they are feeling, would help capture true shooting confidence and heat check decisions.

Pass & Catch Quality Context

The model tracks dribble counts to spot catch and shoot looks, but it completely misses pass placement. Anyone who plays knows that receiving a crisp pass right in your shooting pocket lets you rise up smoothly, while catching a low pass at your ankles completely ruins your rhythm and footwork. Measuring pass speed, pass distance, and how fast a player can gather and release after the catch would better separate clean rhythm shots from scrambled looks.

Defender Physical Profiles & Contest Dimensions

Right now, defender proximity only measures distance on the floor in two dimensions. A six foot guard standing three feet away gives a shooter plenty of vision, but a long six foot eleven wing with a seven foot three wingspan at that same distance creates an absolute wall. Adding player measurements like height, weight, and wingspan, alongside vertical contest tracking, would show the actual visual obstacle a shooter has to shoot over.

Defensive Schemes & Trapping Density

Looking only at the closest defender leaves out what the rest of the defense is doing. A player taking a shot against one defender in single coverage is in a much better spot than a ball handler getting trapped in the corner by a hard double team or driving into a crowded paint. Creating a metric that counts how many defenders are clogging up space nearby would help account for double teams, wall building defense, and secondary help rotations.

Game Flow & Venue Impact

Right now, the model treats a shot in the first two minutes of the first quarter the exact same as a high pressure shot late in the fourth quarter. It misses the energy of the game, like whether a team is on a double digit scoring run or how the home crowd is affecting play. Bringing in play by play data to track game momentum, score differential, and home court advantage would give much needed context to late game shot selection.

5. Conclusion

By pairing real world basketball intuition with non-linear gradient boosting, the model captures defender closeout dynamics, shooter movement, and spatial shot difficulty to achieve a competitive Log-Loss of 0.62922. All code, final predictions, and project assets are fully reproducible via `project_code.R`.